TRIAD Planner: Three-Level Robust Imitation Learning for Autonomous Driving

Jiyun Kim[1], **Ahmad Mouri Zadeh Khaki**[1], **and Kyunghwan Choi**[1,*]
[1]Korea Advanced Institute of Science and Technology (KAIST)
{JiyunKim, ahmadmouri, kh.choi}@kaist.ac.kr

Abstract: Imitation learning for autonomous driving suffers from distribution shift between logged expert trajectories and self-induced deployment states. In Transformer-based planners, this shift can appear as excessive ego-state attention concentration, incomplete scene understanding, and unsafe mode selection under interaction uncertainty. This paper presents a robust imitation-learning planner based on a three-level robustness formulation. 1) Channel-level robustness: constrained ego-state attention to limit excessive concentration on a small subset of state channels. 2) Scene-level robustness: offline vision-language model (VLM) guidance to encourage the ego query to retain interaction-critical agent evidence and route-relevant map evidence. 3) Decision-level robustness: training-time multimodal supervision reshaped with a tail-risk score based on mean excess above Value at Risk. The scene-level VLM guidance and decision-level tail-risk supervision are used only during training, and the deployed planner incurs no additional inference-time overhead. On nuPlan test14, the proposed planner achieves the strongest performance among the evaluated learning-based methods and improves over CAR Planner, a strong constrained-attention imitation-learning baseline. On `test14-hard`, TRIAD Planner achieves NR-CLS and R-CLS of 73.39 and 67.12, compared with CAR Planner's reported 72.94 and 64.90, respectively. Ablation results show that combining all three-level components yields consistent performance across diverse scenarios, supporting the effectiveness of robust imitation learning for autonomous driving.

Keywords: Autonomous Driving, Imitation Learning, Robust Planning, Vision Language Model

1 Introduction

Robust motion planning remains a central challenge in autonomous driving [1, 2]. Imitation learning (IL) learns driving policies from expert demonstrations and has shown competitive performance on modern planning benchmarks [3, 4]. This paradigm allows a planner to learn complex driving behavior directly from data without manually specifying every interaction rule. However, the training signal is typically collected under the logged expert state distribution, while deployment requires the planner to act on states induced by its own previous decisions [5, 6]. Small early deviations can lead the ego vehicle to states where the resulting observations, surrounding-agent configurations, and map-relative poses are rarely encountered in the demonstrations. This distribution shift is especially problematic in closed-loop driving, where prediction errors, representation errors, and trajectory-selection errors can feed back into subsequent decisions.

In Transformer-based IL planners [7], this shift can affect multiple stages of the planning pipeline. At the channel level, ego-state attention may become excessively concentrated on a small subset of channels [8, 9, 10, 11, 12]. At the scene level, attention may under-emphasize interaction-critical agents or route-relevant map context. Prior work on VLM-derived rewards [13] and language-guided planning [14] motivates incorporating semantic guidance to address this concern. At the

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

decision level, a multimodal decoder may assign high likelihood to a trajectory mode that remains close to the logged expert while still having unfavorable tail behavior under interaction uncertainty. These effects arise over different variables, namely ego-state channels, external scene tokens, and candidate trajectory modes. They are therefore difficult to address with a single regularizer or final-stage selector.

This paper formulates robust imitation planning as a *three-level robustness* problem, spanning ego-state aggregation at the channel level, interaction-relevant scene-token weighting at the scene level, and supervision of candidate trajectory modes under interaction risk at the decision level. Based on this formulation, **TRIAD Planner** (**T**hree-level **R**obust **I**mitation learning for **A**utonomous **D**riving) combines constrained ego-state attention, offline VLM-guided scene supervision, and tail-risk-aware mode supervision within a feed-forward planner. The offline VLM-guided scene prior and tail-risk-aware mode targets are used only during training and are disabled during deployment.

TRIAD Planner is evaluated on the nuPlan benchmark [6] using open-loop score (OLS), non-reactive closed-loop score (NR-CLS), and reactive closed-loop score (R-CLS). The main contributions are as follows.

- **A unified three-level robustness framework for imitation-based planning.** Closed-loop imitation degradation in Transformer-based planners is addressed across channel, scene, and decision levels, corresponding to ego-state channel aggregation, external scene-token relevance, and multimodal trajectory-mode selection.
- **Channel- and scene-level robustness in attention-based planning.** CAR Planner's constrained ego-state attention is integrated with offline VLM-guided scene supervision, limiting excessive channel concentration while encouraging the ego query to retain interaction-critical agent and route-relevant map evidence.
- **Decision-level robustness via tail-risk-aware mode supervision.** Clearance-based tail excess shapes hard and soft mode targets, penalizing excess above each mode's own risk quantile during training.
- **Robustness gains on nuPlan across random and hard splits.** Experiments on `test14-random` and `test14-hard` show consistent improvements in open-loop and closed-loop metrics over strong learning-based baselines.

2 Related Work

Autonomous driving planners span rule-based, learning-based, and hybrid formulations. Rule-based methods such as IDM [15] and PDM-Closed [16] encode motion and traffic priors, while learning-based planners such as UrbanDriver [17], GC-PGP [18], and PDM-Open [16] learn policies from driving data. Transformer-based imitation planners improve interaction modeling [11], although they can also suffer from ego-state shortcut dependence. PlanTF [12] mitigates this issue with a State Dropout Encoder, while CAR Planner [19] constrains ego-state attention concentration with an augmented-Lagrangian formulation. These methods motivate channel-level robustness, although they do not directly supervise external scene-token relevance.

Language- and vision-language-based planners improve scene understanding by incorporating high-level reasoning. LLM-Assist [20] uses language-based reasoning for closed-loop planning on nuPlan, and recent VLM-based planners [21, 22] use multimodal inputs for scene reasoning and motion planning. These methods show the value of language-based context, while online large-model inference can increase planning latency. TRIAD Planner instead queries the VLM offline and uses the resulting scene prior only to supervise ego-query attention during training.

Tail-risk objectives have been studied in planning, control, and imitation learning. CVaR provides a standard upper-tail risk formulation [23], with applications in risk-constrained control [24] and adversarial imitation learning [25]. In multimodal imitation planning, supervision additionally requires selecting among candidate trajectory modes. TRIAD Planner uses excess above mode-specific risk quantiles to construct training-time hard and soft mode targets.

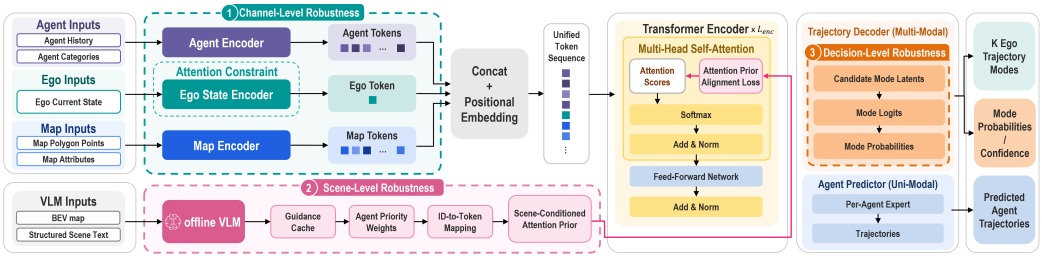


Figure 1: Overview of TRIAD Planner with channel-, scene-, and decision-level robustness components.

3 TRIAD Planner

3.1 Planner Overview

As shown in Figure 1, TRIAD Planner receives the current ego state, surrounding-agent histories, map polygon points and attributes, and outputs K candidate ego trajectories with mode logits. The base planner is augmented with three robustness components. The channel-level component constrains ego-state attention before scene-token fusion, the scene-level component supervises ego-query attention over external tokens, and the decision-level component reshapes multimodal supervision with a tail-risk score from predicted ego-agent clearance.

3.2 Base Architecture

Let $\mathbf{s}^{\text{ego}} \in \mathbb{R}^6$ denote the current ego state, represented by (x, y, ψ, v, a, s) for position, heading, velocity, acceleration, and steering angle. Let $\{\mathbf{H}_i\}_{i=1}^{A}$ and $\{\mathbf{P}_j\}_{j=1}^{M}$ denote the valid agent-history and map-polygon inputs, where A and M are the numbers of valid agent and map tokens after masking. Agent and map inputs are encoded as tokens in $\mathbb{R}^d$. The ego token is built by channel-wise projection followed by learned-query aggregation. A shared Transformer encoder processes all tokens. The final ego token is decoded into K candidate ego trajectories and mode logits.

3.3 Channel-Level Robustness Component via Constrained Ego-State Attention

We adopt CAR Planner's ego-state attention constraint for channel-level robustness [19]. Each channel of $\mathbf{s}^{\text{ego}} \in \mathbb{R}^6$ is embedded independently, and a learned query attends to the six channel embeddings. Let $a_\theta^{(n)} \in \Delta^{C-1}$ denote the head-averaged ego-state attention weights for sample n, where $C = 6$. The sample-wise deviation from uniform attention and its batch average are

$$d(a_\theta^{(n)}) = \frac{1}{C}\sum_{i=1}^{C}\left|a_{\theta,i}^{(n)} - \frac{1}{C}\right|, \qquad D(\theta) = \frac{1}{N}\sum_{n=1}^{N} d(a_\theta^{(n)}).$$

The constraint $D(\theta) \le \delta$ regulates attention concentration averaged over the training minibatch while permitting non-uniform channel weights.

Following CAR Planner, we use projected ALM with $g(\theta) = D(\theta) - \delta$:

$$\mathcal{L}_{\text{ALM}} = \frac{[\mu + \rho g(\theta)]_+^2 - \mu^2}{2\rho}. \tag{1}$$

Here, $[z]_+ = \max(0, z)$, $\mu \ge 0$ is initialized at zero, and $\rho > 0$. After each optimizer step, $\mu_{t+1} = [\mu_t + \rho g(\theta_t)]_+$ uses the same forward-pass residual.

This regularizer acts only on ego-state aggregation before token fusion. It regulates channel attention allocation, while scene-level guidance supervises relevance over external tokens.

3.4 Scene-Level Robustness Component via VLM-Guided Token Relevance

Scene-level robustness addresses irrelevant attention weights assigned to external agent and map tokens. For training samples with valid VLM guidance, TRIAD Planner constructs an offline VLM-guided scene prior from scene-conditioned agent priority weights. This prior serves as a training target for the ego-query attention distribution.

3.4.1 Offline Multimodal Guidance Generation

For each guided training sample, GPT-5.4-Mini is queried offline using a fixed prompt without manual annotation. Each input consists of an ego-centric BEV rendering and structured scene text, as illustrated in Figure 2. The BEV rendering contains surrounding agents, identifiers, velocity cues, traffic-light states, and map elements, while the structured text summarizes ego dynamics, route context, nearby agents, and the target maneuver derived from the target trajectory.

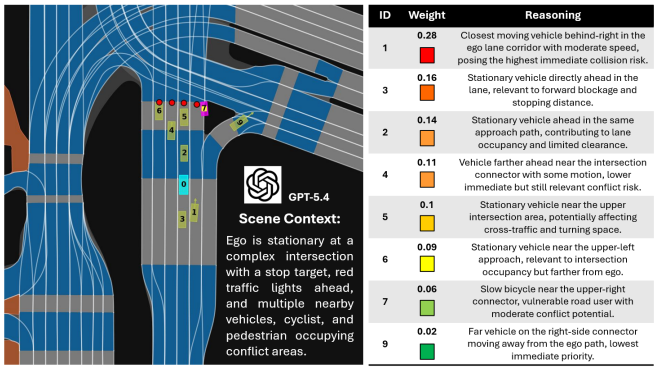


Figure 2: Offline VLM-guided scene-prior construction.

The prompt asks the VLM to identify agents whose route conflict or interaction risk is relevant to the maneuver, thereby prioritizing agents according to their intended-path relevance. The VLM returns constrained JSON priority weights together with short textual reasons. The priorities are cached before training, and the reasons are retained for qualitative inspection.

3.4.2 Teacher Prior Construction and Attention Supervision

For guided samples, invalid agent IDs are discarded, and the remaining valid agent priorities are renormalized. Map polygons are then scored with a route-aware geometric prior, and the agent and map scores are combined and normalized over valid external tokens to obtain $\mathbf{p}^{\text{scene}}$, defined over the union of valid agent and map tokens. Let $\mathcal{B}_{\text{guide}}$ denote the subset of minibatch samples with valid VLM guidance, and let $\widehat{\mathbf{p}}^{\text{scene}}$ denote the masked and normalized attention distribution of the ego query over valid agent and map keys. The scene-level guidance loss is

$$\mathcal{L}_{\text{scene}} = -\frac{1}{|\mathcal{B}_{\text{guide}}|} \sum_{n \in \mathcal{B}_{\text{guide}}} \sum_{u=1}^{A+M} p_{n,u}^{\text{scene}} \log \widehat{p}_{n,u}^{\text{scene}}. \tag{2}$$

This loss encourages the ego query to retain interaction-critical dynamic agents while preserving route-relevant map evidence.

3.5 Decision-Level Robustness Component via Tail-Risk-Aware Mode Supervision

For each predicted mode k, the clearance-risk sequence $z^{(k)} = (z_1^{(k)}, \ldots, z_{T_f}^{(k)})$ is computed over T_f future steps from predicted ego-agent clearance deficits, accounting for the effective safety radius and a safety margin.

Let $\eta_\alpha^{(k)} = \text{VaR}_\alpha(z^{(k)})$ denote the mode-specific empirical Value-at-Risk (VaR) at quantile level $\alpha \in (0, 1)$. The tail-excess score is

$$r_k^{\text{tail}} = \frac{1}{(1-\alpha)T_f} \sum_{t=1}^{T_f} \left[z_t^{(k)} - \eta_\alpha^{(k)} \right]_+ . \tag{3}$$

This score summarizes upper-tail deviations in clearance risk relative to each mode's own quantile. It provides a training signal that penalizes larger exceedances when constructing mode targets.

Let $p^{\text{mode}} = \text{softmax}(\pi)$ denote the predicted mode distribution. During training, the supervised mode is selected by combining mode confidence and tail excess as

$$s_k = -\log p_k^{\text{mode}} + \lambda_{\text{tail}} r_k^{\text{tail}}, \qquad k^* = \arg\min_k s_k. \tag{4}$$

Here, λ_{tail} balances mode confidence against tail excess. The selected mode is regressed toward the logged expert trajectory.

A risk-aware soft target over modes is also constructed as

$$q_k^{\text{mode}} = \frac{\text{sg}(p_k^{\text{mode}}) \exp(-\lambda_{\text{tail}} \tilde{r}_k^{\text{tail}})}{\sum_{j=1}^{K} \text{sg}(p_j^{\text{mode}}) \exp(-\lambda_{\text{tail}} \tilde{r}_j^{\text{tail}})}, \tag{5}$$

where $\tilde{r}_k^{\text{tail}}$ is the within-sample normalized tail-excess score and $\text{sg}(\cdot)$ denotes stop-gradient. The probability head is trained with

$$\mathcal{L}_{\text{risk}} = \text{KL}(\mathbf{q}^{\text{mode}} \| \mathbf{p}^{\text{mode}}).$$

This loss trains the predicted mode distribution to match the risk-aware soft targets.

3.6 Training Objective and Inference Procedure

Training combines a base imitation objective with the three robustness terms. Using the risk-aware mode k^* from Eq. 4, the standard imitation objective combines trajectory regression, mode classification, and auxiliary agent prediction:

$$\mathcal{L}_{\text{base}} = \mathcal{L}_{\text{traj}} + \lambda_{\text{mode}} \mathcal{L}_{\text{mode}} + \lambda_{\text{pred}} \mathcal{L}_{\text{pred}}. \tag{6}$$

Here, $\mathcal{L}_{\text{traj}}$ supervises the selected ego trajectory, $\mathcal{L}_{\text{mode}}$ trains the mode logits with the selected mode target, and $\mathcal{L}_{\text{pred}}$ supervises surrounding-agent prediction.

The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{base}} + \mathcal{L}_{\text{ALM}} + \lambda_{\text{scene}} \mathcal{L}_{\text{scene}} + \lambda_{\text{risk}} \mathcal{L}_{\text{risk}}, \tag{7}$$

Here, λ_{risk} weights the risk-supervision loss, while λ_{tail} controls the risk sensitivity used to construct its targets. At inference, the final trajectory is selected from the learned mode distribution in a standard feed-forward pass.

4 Experiments

4.1 Experimental Setup

TRIAD Planner was trained on the nuPlan v1.1 training split using the `training_scenarios_1M` filter. No additional scenario, city, or map filtering was applied.

VLM guidance was generated for approximately 20% of the training set to limit offline processing cost, and guided samples were oversampled with probability 0.5 during training. The planners were trained for 25 epochs with batch size 64. Training used AdamW with an initial learning rate of $1\text{e}{-3}$, weight decay $1\text{e}{-4}$, and a cosine schedule with 3 warm-up epochs. State perturbation augmentation [26] was applied with probability 0.5. The current ego state was perturbed with bounded noise, and all coordinates were re-normalized in the perturbed ego frame.

All experiments were conducted on a workstation equipped with an AMD EPYC 9354 32-Core Processor, 251 GiB of system memory, and four NVIDIA RTX PRO 5000 GPUs.

Evaluation follows the nuPlan benchmark on `test14-random` and `test14-hard`. Both splits cover 14 scenario types. The reported metrics are open-loop score (OLS), non-reactive closed-loop score (NR-CLS), and reactive closed-loop score (R-CLS). OLS measures imitation quality against logged expert behavior. NR-CLS evaluates closed-loop rollout with replayed agents. R-CLS evaluates closed-loop rollout with reactive simulated agents.

Table 1: Performance comparison on test14. CAR Planner scores are reported results [19].

Planners		Test14-random			Test14-hard		
Type	**Method**	OLS	NR-CLS	R-CLS	OLS	NR-CLS	R-CLS
Expert	Log-replay	100.0	94.03	75.86	100.0	85.96	68.80
Rule-based	IDM	34.15	70.39	72.42	20.07	56.15	62.26
	PDM-Closed	46.33	90.05	91.64	26.43	65.08	75.19
Hybrid	PDM-Hybrid	81.26	90.10	91.29	74.08	65.99	76.07
Learning-based	RasterModel	62.93	68.71	67.52	52.38	50.65	51.41
	UrbanDriver	82.45	63.57	60.92	76.90	51.67	49.06
	GC-PGP	80.89	61.57	54.88	75.40	47.16	41.39
	PDM-Open	84.14	52.81	57.23	79.06	33.51	35.83
	PlanTF	86.27	85.23	79.36	83.34	70.03	59.83
	Pluto (w/o post.)	87.15	86.60	77.96	81.89	70.28	56.70
	CAR Planner	87.64	85.25	79.15	86.09	72.94	64.90
	TRIAD Planner	89.72	87.65	82.55	87.13	73.39	67.12

Table 2: Scenario-level inference latency on 261 evaluation scenarios. Lower is better. Header abbreviations denote UrbanDriver, RasterModel, PDM-Open, PDM-Hybrid, and PDM-Closed.

Metric	CAR	TRIAD	PlanTF	Urban	Raster	PDM-O	GC-PGP	PDM-H	PDM-C	Pluto
Median (s)	**14.97**	15.11	15.15	17.09	18.07	19.05	21.76	32.65	34.32	45.39
Mean (s)	**15.00**	15.18	15.01	18.63	18.05	19.31	21.64	32.74	34.52	43.48
Std. (s)	2.45	2.44	**2.43**	6.27	6.31	6.86	6.89	8.44	9.09	13.89
Total (min)	**65.2**	66.0	65.3	81.1	78.5	84.0	94.1	142.4	150.2	189.1

4.2 Experimental Results

Overall benchmark results. Table 1 compares TRIAD Planner with rule-based, hybrid, and learning-based planners on `test14-random` and `test14-hard`. Rule-based and hybrid planners achieve strong closed-loop scores on `test14-random`, but exhibit substantially lower OLS and larger performance drops on `test14-hard`. Although learning-based planners remain vulnerable to closed-loop distribution shift, they offer a complementary approach to generalizing across diverse scenarios without relying entirely on hand-crafted rules. Pluto [27] is reported without post-processing for a like-for-like comparison of learned outputs.

Among the evaluated learning-based planners, TRIAD Planner obtains the strongest performance across OLS, NR-CLS, and R-CLS. Compared with CAR Planner's reported results, TRIAD Planner achieves higher R-CLS: 82.55 versus 79.15 on `test14-random` and 67.12 versus 64.90 on `test14-hard`, with higher OLS and NR-CLS on both splits. This suggests that scene-level guidance and decision-level supervision improve closed-loop robustness beyond constrained ego-state aggregation.

Inference efficiency. Table 2 summarizes scenario-level inference latency. TRIAD Planner exhibits mean runtimes and runtime variability comparable to CAR Planner and PlanTF. These measurements are consistent with applying scene and decision supervision during training while retaining standard feed-forward inference.

Ablation analysis of the three levels. Figure 3 evaluates how the channel (C), scene (S), and decision (D) components contribute to the Base planner. Base denotes the vanilla Transformer planner, and Base+C denotes the channel-only CAR Planner reference. Scene- and decision-level components affect the Base planner through different supervision targets, with S guiding external-token relevance and D reshaping risk-aware mode selection. The individual Base+S and Base+D variants improve several closed-loop scores over Base, suggesting that both external-token relevance and mode-level risk provide useful training signals.

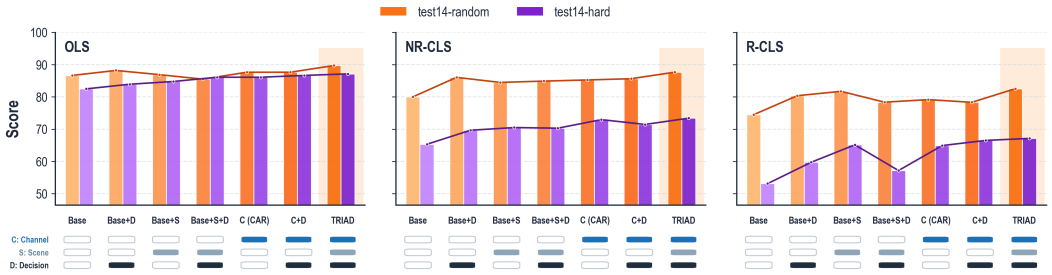


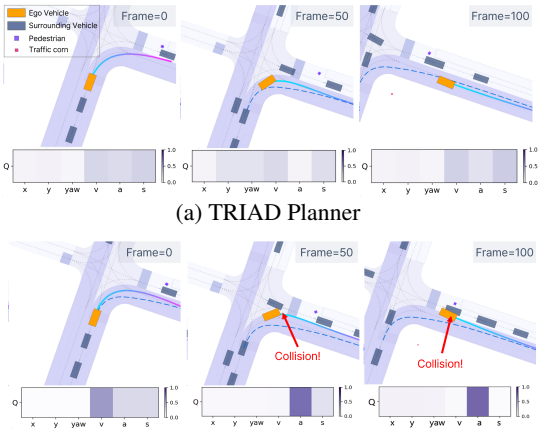
Figure 3: Channel (C), scene (S), and decision (D) component comparison on `test14-random` and `test14-hard`. The CAR reference uses reported scores. Higher is better.

Table 3: Comparison of scene priors on test14 and val14. Higher is better.

Method	**test14-random**			**test14-hard**			**val14**		
	OLS	NR-CLS	R-CLS	OLS	NR-CLS	R-CLS	OLS	NR-CLS	R-CLS
PlanTF	86.27	85.23	79.36	83.34	70.03	59.83	88.54	84.73	74.85
heuristic_distance	88.55	86.01	80.76	85.94	69.74	63.99	**89.86**	83.90	79.15
heuristic_future	89.02	**88.38**	81.94	86.14	70.88	60.39	89.85	85.75	78.16
TRIAD Planner	**89.72**	87.65	**82.55**	**87.13**	**73.39**	**67.12**	**89.86**	**85.81**	**80.55**

Base+S+D yields lower R-CLS than either Base+S or Base+D on both splits. On `test14-hard`, its higher OLS relative to Base+S accompanies lower R-CLS, showing different effects on open-loop imitation and reactive closed-loop performance. TRIAD Planner, which combines C, S, and D, achieves the highest OLS, NR-CLS, and R-CLS among the configurations in Figure 3 on both splits. These results support the combined three-level formulation.

Channel-level analysis. Figure 4 visualizes ego-state attention in the same right-turn scenario. At the beginning of the rollout, both planners attend to multiple ego-state channels. After the interaction begins, PlanTF places most of the ego-query attention on the acceleration channel, as shown in Figure 4(b). This concentration coincides with trajectory drift toward the nearby vehicle and a collision in this example. In contrast, TRIAD Planner distributes attention more broadly across dynamic channels, including velocity and acceleration, while remaining closer to the route centerline.



(a) TRIAD Planner

(b) PlanTF

Figure 4: Ego-state encoder attention in a right-turn scenario for (a) TRIAD Planner and (b) PlanTF.

Scene-level analysis. To evaluate whether the scene-level supervision captures information beyond simple proximity cues, the offline VLM-guided scene prior is replaced with two heuristic priors while keeping the remaining training and inference protocol unchanged. The distance-to-ego prior ranks agents according to their current distance from the ego vehicle, whereas the future-path prior ranks agents according to their proximity to the expert future trajectory. Expert future information is used only for offline teacher construction, allowing the future-path prior to test whether the VLM-guided prior merely follows the expert path.

Figure 5 visualizes the three scene priors, while Table 3 reports their performance on test14 and val14. Both heuristic priors improve over PlanTF across the reported metrics. On `test14-hard`, the VLM-guided prior outperforms both heuristic priors across all three metrics. Its R-CLS advantage over the future-path prior highlights the benefit of scene-conditioned guidance under reactive evaluation. On `test14-random`, it achieves the highest OLS and R-CLS, while the future-path prior achieves the highest NR-CLS. On val14, which contains approximately four times more scenarios than the test14 splits, TRIAD Planner is best or tied-best on all metrics. These comparisons support the use of scene-conditioned VLM guidance for agent prioritization.

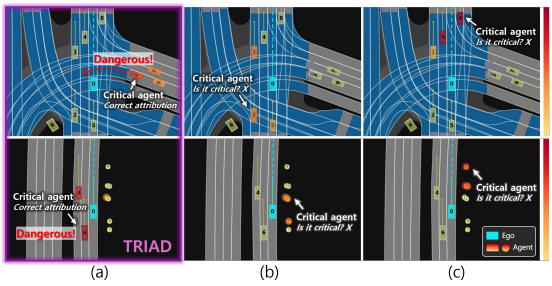


Figure 5: Scene-level prior visualizations. Warmer colors denote higher agent priority. (a) TRIAD Planner's offline VLM-guided scene prior. (b) Heuristic distance-to-ego prior. (c) Heuristic future-path prior.

Figure 6 compares TRIAD Planner and CAR Planner by aggregating pre-decoder Transformer attention into `Agent`, `Ego`, and `Map` query-key groups.

The VLM-guided scene prior supervises the ego query over external agent and map tokens, so the analysis focuses on the `Ego` query row. Both planners remain map-oriented, reflecting the importance of lane and route geometry. TRIAD Planner increases ego-to-agent attention from 0.168 to 0.198 while maintaining dominant ego-to-map attention, which changes only slightly from 0.819 to 0.802. Ego-to-ego attention decreases from 0.013 to approximately zero, indicating a weaker ego-token self-loop after ego-state features have already been encoded before token fusion.

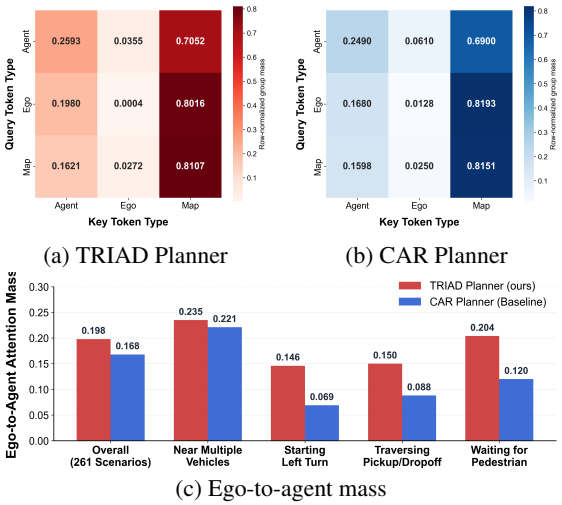
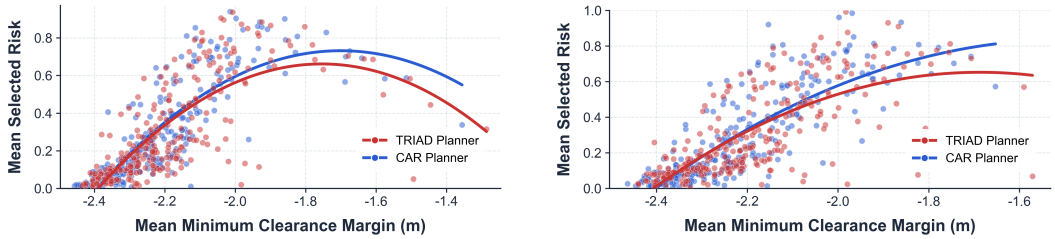


Figure 6: Scene-level attention analysis on `test14-random`. (a) and (b) show group attention maps, and (c) shows `Ego`→`Agent` attention mass.

Figure 6(c) further shows that ego-to-agent attention changes are larger in actor-dependent contexts such as starting left turns, pickup and dropoff zones, and pedestrian-crossing scenes. The increase is more pronounced when waiting for pedestrians than in scenes with multiple nearby vehicles. These trends align with the scene-level objective, which encourages the ego query to retain interaction-critical agents and route-relevant map evidence.

Decision-level analysis. Figure 7 compares the selected-mode tail-excess score against the mean minimum clearance margin after subtracting the effective safety radius. More negative clearance values indicate tighter proximity, while lower scores indicate smaller upper-tail deviations relative to the selected mode's risk quantile.

On both splits, TRIAD Planner exhibits a lower fitted tail-excess trend than CAR Planner over most of the observed clearance range. At comparable clearance margins, the lower fitted scores indicate smaller upper-tail deviations relative to each selected mode's own risk quantile. These trends suggest that training-time decision-level supervision encourages the learned mode distribution to favor lower-tail-excess candidates under similar proximity conditions.



(a) `test14-random` (b) `test14-hard`

Figure 7: Selected-mode tail-excess trends over clearance margins on `test14-random` and `test14-hard`. Lower scores indicate smaller exceedances above the mode-specific VaR.

5 Conclusion

This paper presented TRIAD Planner, a three-level robustness framework for imitation-learning-based autonomous driving. The planner combines CAR Planner's constrained ego-state attention with offline VLM-guided scene supervision and tail-risk-aware mode supervision. The scene and decision objectives shape attention and mode selection during training, while deployment retains the feed-forward planning architecture.

Experiments on both nuPlan test14 splits showed improvements in OLS, NR-CLS, and R-CLS over the learning-based baselines in the main comparison. Component comparisons favored the full three-level formulation, and the VLM-guided prior outperformed both geometric teacher priors across all three metrics on `test14-hard`. Together, these results support jointly addressing ego-state attention, scene-token relevance, and trajectory-mode supervision in imitation-based planning.

6 Limitations

Guidance was generated for approximately 20% of the cached scenario-token corpus. Approximately 3% of the VLM outputs had raw priority weights that summed to more than one. These outputs were filtered or renormalized before training. Omitted or underweighted interaction-critical agents and incomplete BEV-text descriptions may still introduce noisy supervision, particularly in rare or ambiguous interactions.

The guidance can cover multiple scenario tokens. It does not provide timestep-level temporal supervision and may miss agents or interactions whose relevance emerges over longer temporal context.

Evaluation is limited to nuPlan, a standard large-scale benchmark for imitation-learning planner generalization. Cross-dataset evaluation on additional datasets and simulators remains future work.

Acknowledgments

This work was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025-00554087), and in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) under the Leading Generative AI Human Resources Development program (RS-2026-25544188), funded by the Korea government (MSIT).

References

[1] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad. A survey of end-to-end driving: Architectures and training methods. *IEEE Transactions on Neural Networks and Learning Systems*, 33(4):1364–1384, 2022.

[2] K. Muhammad, A. Ullah, J. Lloret, J. Del Ser, and V. H. C. De Albuquerque. Deep learning for safe autonomous driving: Current challenges and future directions. *IEEE Transactions on Intelligent Transportation Systems*, 22(7):4316–4336, 2021.

[3] D. A. Pomerleau. Alvinn: An autonomous land vehicle in a neural network. *Advances in Neural Information Processing Systems*, 1, 1988.

[4] M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi. A survey of imitation learning: Algorithms, recent developments, and challenges. *IEEE Transactions on Cybernetics*, 54(12): 7173–7186, 2024.

[5] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

[6] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.

[7] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. TransFuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):12878–12895, 2023.

[8] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[9] C.-C. Chuang, D. Yang, C. Wen, and Y. Gao. Resolving copycat problems in visual imitation learning via residual action prediction. In *European Conference on Computer Vision*, pages 392–409. Springer, 2022.

[10] C. Wen, J. Lin, T. Darrell, D. Jayaraman, and Y. Gao. Fighting copycat agents in behavioral cloning from observation histories. *Advances in Neural Information Processing Systems*, 33: 2564–2575, 2020.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[12] J. Cheng, Y. Chen, X. Mei, B. Yang, B. Li, and M. Liu. Rethinking imitation-based planners for autonomous driving. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14123–14130. IEEE, 2024.

[13] Z. Huang, Z. Sheng, Y. Qu, J. You, and S. Chen. VLM-RL: A unified vision language models and reinforcement learning framework for safe autonomous driving. *Transportation Research Part C: Emerging Technologies*, 180:105321, 2025.

[14] Y. Chen, Z.-h. Ding, Z. Wang, Y. Wang, L. Zhang, and S. Liu. Asynchronous large language model enhanced planner for autonomous driving. In *European Conference on Computer Vision*, pages 22–38. Springer, 2024.

[15] M. Treiber, A. Hennecke, and D. Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical Review E*, 62(2):1805–1824, 2000.

[16] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Conference on Robot Learning*, pages 1268–1281. PMLR, 2023.

[17] O. Scheel, L. Bergamini, M. Wolczyk, B. Osiński, and P. Ondruska. Urban Driver: Learning to drive from real-world demonstrations using policy gradients. In *Conference on Robot Learning*, pages 718–728. PMLR, 2022.

[18] M. Hallgarten, M. Stoll, and A. Zell. From prediction to planning with goal conditioned lane graph traversals. In *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, pages 951–958. IEEE, 2023.

[19] J. Kim and K. Choi. CAR Planner: Constrained-attention-based robust imitation learning for autonomous driving. *Authorea Preprints*, 2025.

[20] S. P. Sharan, F. Pittaluga, V. Kumar B. G., and M. Chandraker. LLM-Assist: Enhancing closed-loop planning with language-based reasoning. *arXiv preprint arXiv:2401.00125*, 2023.

[21] Y. Zheng, Z. Xing, Q. Zhang, B. Jin, P. Li, Y. Zheng, Z. Xia, Y. Chen, and D. Zhao. PlanAgent: A multimodal large language agent for closed-loop vehicle motion planning. *IEEE Transactions on Cognitive and Developmental Systems*, 18(4):1113–1126, 2026.

[22] Z. Tang, S. Zhang, J. Deng, C. Wang, G. You, Y. Huang, X. Lin, and Y. Zhang. VLMPlanner: Integrating visual language models with motion planning. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 5040–5049, 2025.

[23] R. T. Rockafellar and S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2:21–42, 2000.

[24] A. Hakobyan, G. C. Kim, and I. Yang. Risk-aware motion planning and control using CVaR-constrained optimization. *IEEE Robotics and Automation Letters*, 4(4):3924–3931, 2019.

[25] A. Santara, A. Naik, B. Ravindran, D. Das, D. Mudigere, S. Avancha, and B. Kaul. RAIL: Risk-averse imitation learning. *arXiv preprint arXiv:1707.06658*, 2017.

[26] M. Bansal, A. Krizhevsky, and A. Ogale. ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst. *arXiv preprint arXiv:1812.03079*, 2018.

[27] J. Cheng, Y. Chen, and Q. Chen. Pluto: Pushing the limit of imitation learning-based planning for autonomous driving. *arXiv preprint arXiv:2404.14327*, 2024.

A VLM-Based Guidance Generation

A.1 Multimodal Input

The bird's-eye-view (BEV) image is rendered using the nuPlan map API in a $100\,\mathrm{m} \times 100\,\mathrm{m}$ local window at a resolution of 1024×1024, with the ego vehicle centered and its heading pointing upward.

The structured description is constructed from cached nuPlan features and includes the nine nearest agents. The target maneuver is derived from the accumulated heading change along the logged target trajectory. The input fields are illustrated in the example below.

Textual input example. The following example shows the structured scene description paired with the BEV image.

```
{
  "scenario": {"scenario_type": "stationary"},
  "ego": "The ego vehicle's speed is 0.0015 m/s with acceleration of
          0.0099 m/s^2. The steering angle is 0.0 degrees and the yaw
          rate is 0.0 degrees/s.",
  "agents": "There are 9 agents near the ego vehicle:
          1: VEHICLE, distance 3.81 m, speed 4.36 m/s;
          2: VEHICLE, distance 10.24 m, speed 0.0 m/s;
          3: VEHICLE, distance 8.23 m, speed 0.0 m/s;
          4: VEHICLE, distance 19.16 m, speed 2.11 m/s;
          5: VEHICLE, distance 20.21 m, speed 0.0 m/s;
          6: VEHICLE, distance 22.92 m, speed 0.0 m/s;
          7: BICYCLE, distance 23.21 m, speed 0.09 m/s;
          8: PEDESTRIAN, distance 23.27 m, speed 0.01 m/s;
          9: VEHICLE, distance 29.52 m, speed 6.3 m/s.",
  "map": "The typical speed limit is approximately 11.93 m/s.",
  "target_trajectory": "Stop."
}
```

A.2 Prompt Design

The offline teacher uses `gpt-5.4-mini` through the Chat Completions API with temperature 0. No maximum completion-token limit is explicitly set. The prompt template is reproduced below with per-sample fields shown as placeholders and formatting normalized for readability.

```
[System]
You are an expert autonomous driving system analyst.

[User]
You are given a bird's-eye-view (BEV) map image.
The map is ego-centric. The ego vehicle is always at the center of the image.
The map is heading-up: the ego vehicle's forward driving direction always
points toward the top of the image.
Therefore, left/right in the image correspond directly to left/right of the
ego vehicle, and the top/bottom of the image correspond to ahead/behind the
ego vehicle.

The color legend for the BEV map is as follows:
- Ego vehicle: cyan (#00ffff) with the label of 0
- Vehicles: yellow-green (#b0bb4a)
- Cyclists: magenta (#ff00ff)
- Pedestrians: yellow squares (#ffff33)
- Regular road lanes: gray (#7a7a7a)
- Intersection area: dark gray (#3a3a3a)
- Lane connectors / turning paths: blue (#115d91)
- Crosswalks: beige (#c8b89a) with white dashed hatching
- Sidewalks / walkways: brown (#a0522d)
```

```
- Parking areas: dark navy (#021e30)
- Traffic lights: colored circles (green (#00ff00), yellow (#ffff00),
  red (#ff0000), or gray (#aaaaaa)) at the entry of lane connectors.

Each agent box has an ID label and a velocity arrow showing its direction
and speed. Analyze the current driving scene based on the BEV map and the
state information.

State information:
- Scenario: {scenario}
- Ego: {ego}
- Agents: {agents}
- Map: {map}
- Target trajectory: {target_trajectory}

I need to use a guidance factor to represent the weighting characteristics of
agents and scene context in autonomous driving for avoiding collision and
off-road trajectory.

agents_priority: For each identified agent in the scene excluding the ego
vehicle, report its priority weight based on its type, position, navigation
relevance, and distance to the ego vehicle. Higher weights represent higher
priority. Normalize the weights so that their sum is 1.

Output only a valid Python dictionary. Use exactly these keys and value
formats:
{
  "agents_priority": {
    "AGENT_ID": {"weight": float.2f, "reason": "string"}
  }
}
```

Here, `float.2f` is schema notation; returned weights are numeric literals represented to two decimal places.

Output example. For the textual input above, the teacher returns priorities and short rationales in the requested dictionary format.

```
{
  "agents_priority": {
    "1": {"weight": 0.28,
          "reason": "Closest moving vehicle ahead-right in the ego lane
                     corridor, posing the highest immediate collision risk."},
    "3": {"weight": 0.16,
          "reason": "Stationary vehicle directly ahead in the lane."},
    "2": {"weight": 0.14,
          "reason": "Stationary vehicle ahead in the same approach path."},
    "4": {"weight": 0.11,
          "reason": "Vehicle farther ahead near the intersection connector."},
    "5": {"weight": 0.10,
          "reason": "Stationary vehicle near the intersection area."},
    "6": {"weight": 0.09,
          "reason": "Stationary vehicle near the upper-left approach."},
    "7": {"weight": 0.06,
          "reason": "Slow bicycle near the upper-right connector."},
    "8": {"weight": 0.04,
          "reason": "Pedestrian near the upper-right area."},
    "9": {"weight": 0.02,
          "reason": "Far vehicle moving away from the ego path."}
  }
}
```

A.3 Output Validation and Scene-Prior Construction

The returned dictionary is structurally validated and serialized as JSON. API, parsing, or formatting failures are logged, and the corresponding samples are reprocessed offline. Agent identifiers are matched to the agents shown in the BEV representation; invalid identifiers and padded agents are ignored. The remaining weights are converted to floating-point values, clipped to $[0, 1]$, and combined with the route-aware map scores before normalization over valid external tokens. If no usable VLM agent priority remains, inverse-distance agent scores provide the fallback prior.

Guidance generation focuses on interaction- and maneuver-related scenarios. During training, the guided-sample fraction is controlled through weighted sampling with replacement.